

Android Malware Detection

Using Machine Learning

Abstract:

Android devices are often targeted by malware, which can steal data, slow down devices, and cause financial or privacy problems. Traditional antivirus methods cannot always detect new or hidden malware. This project uses machine learning (ML) to detect Android malware. By analyzing features like app permissions, API calls, network activity, and system behavior, the ML model can classify apps as safe or malicious. This approach helps detect unknown malware and improves device security.

Problem Statement:

Android malware continues to evolve, exploiting system vulnerabilities, permissions, and APIs to perform malicious activities. Existing signature-based detection methods fail to detect new or polymorphic malware efficiently. This creates a need for intelligent, data-driven methods capable of analyzing application behavior and detecting malicious activity with high accuracy. Machine learning provides an effective solution for detecting both known and unknown malware by learning patterns from historical app data.

Objectives:

- To analyze Android application behavior using machine learning
- To design a malware detection model capable of classifying apps as benign or malicious
- To compare performance of different ML algorithms (e.g., KNN, Decision Tree, Random Forest, SVM)
- To improve detection accuracy over traditional signature-based methods
- To develop a scalable, interpretable, and automated malware detection system

Introduction:

The Android platform dominates the global mobile market, making it an attractive target for malware attacks. Malware can perform unauthorized operations such as data theft, financial fraud, and device hijacking. Traditional antivirus solutions rely on known malware signatures and struggle to detect new or obfuscated threats. Machine learning offers a solution by learning patterns from historical app data and predicting the likelihood of malicious behavior. Features such as API calls, requested permissions, system calls, network activity, and app behavior metrics can be used to train ML models. This project uses supervised ML algorithms to classify Android applications as benign or malicious, improving the speed and accuracy of detection.

Existing System:

- Signature-based detection using antivirus databases
- Limited ability to detect new, unknown, or obfuscated malware
- Relies on predefined malware patterns and rules
- Poor performance in real-time dynamic malware scenarios

Limitations of Existing System:

- Unable to detect previously unseen malware effectively
- High dependency on frequent signature updates
- Cannot adapt to complex malware behavior patterns
- Limited support for automated real-time analysis

Proposed System:

The proposed system uses machine learning to detect Android malware. Key features from applications, such as permissions, API calls, network activity, and system calls, are extracted and preprocessed. Multiple ML algorithms (KNN, Decision Tree, Random Forest, SVM) are trained on historical malware and benign app datasets. When a new application is analyzed, the trained model predicts whether it is benign or malicious. This system offers a data-driven, intelligent, and scalable approach to malware detection, capable of identifying previously unseen malware variants.

Key Components:

- **Data Collection:** Malware and benign app datasets (e.g., Drebin, AndroZoo)
- **Data Preprocessing:** Cleaning, normalization, and encoding of features
- **Feature Extraction:** Permissions, API calls, network activity, system calls
- **Machine Learning Models:** KNN, Decision Trees, Random Forest, SVM
- **Prediction Module:** Classifying applications as benign or malicious
- **Evaluation Metrics:** Accuracy, Precision, Recall, F1-score

Advantages:

- Detects both known and unknown malware
- Adaptable to evolving malware patterns
- Interpretable and scalable models
- Reduces dependency on signature updates

Disadvantages:

- Performance depends on quality and size of datasets
- Some algorithms may require high computational resources
- Model selection and hyperparameter tuning are critical
- Feature selection plays a major role in detection accuracy